

docs_chain

同期から外れる文書は存在しない — コンテンツハッシュ化、双方向、アトミック

概要

Docs Chainは、汎用的なGo実装による双方向の文書・データベース依存関係伝播エンジンです。登録されたチェーン内のいずれかの要素が変更されると—Markdownソース、HTML/PDF/DOCX形式のエクスポート、あるいはSQLiteデータベース—コンテンツハッシュによってその変更を検知し、接続された全ての要素に対してアトミックに伝播します。

短い説明

文書とデータベースを同期させるGoエンジン。DAG（カーンのトポロジカル順序、早期カットオフ、双方向同期エッジ、アトミックリネーム＋SQLiteトランザクションによるコミット）上で、Salsaスタイルのコンテンツハッシュ化によるインクリメンタル再計算を行い、リンクされた成果物に変更があればエクスポートを再生成します。

長い説明

Docs Chainは、「Markdownが変更されたらPDFを再生成する」という脆弱なシェルスクリプトを何度も書き直した末に生まれたものです。手作りの同期用接着剤の全てを、本格的なエンジンに置き換えます。プロジェクトの文書やデータベースをチェーンの要素としてモデル化し、いずれかの要素に変更があれば、宣言された全ての方向にその変更を伝播—再生成と再エクスポートをアトミックに行い、追跡対象の成果物が同期から外れることは決してありません。その設計は、スクリプトではなくインクリメンタルビルドシステムの世界から厳密さを借用しています。変更検知は**mtime**ではなくコンテンツハッシュで行われるため、touchによる更新では何も起こらず、1バイトの編集でも必要な再ビルドのみが正確に実行されます—誤検知も見逃しありません。これを一行で厳密に表現すれば、DAG上でのSalsaスタイルのコンテンツハッシュ化によるインクリメンタル再計算であり、カーンのトポロジカル順序、変更のないサブツリーを刈り取る早期カットオフ、宣言された権限に基づく双方向のsyncエッジ、そしてアトミックリネームとSQLiteトランザクションコミットにより、伝播中のクラッシュでも不完全なエクスポートが残ることはありません。

basic-digitalのサブモジュールとして提供され、HelixConstitutionサブモジュールのコア部分として利用されるため、憲法を採用したプロジェクトはDocs Chainを標準装備し、各コンテキストのYAMLを通じて独自のチェーンを登録します。実装の状況は憲法§11.4.6に基づき正直に公開されています：フェーズ1〜4（コアDAG+ハッシュ化、ノードアダプタ/変換、アトミック性を持つ伝播オーケストレータ、設定駆動のマルチコンテキストCLIとsync/verify/doctor/graph/watch）は実装・テスト済み。フェーズ4bでは汎用的な双方向md-to-sqlite/sqlite-to-md組み込み機能（純粋Go、行レベルのドリフト検知、バイト安定のラウンドトリップ）とcolorize-html組み込み機能が追加されます。フェーズ5の包括的な実バイナリエンドツーエンドは実装済みでGREEN。フェーズ6〜7（憲法の配布、ATMOSphereとの連携）は計画段階で、オペレータによるゲート制御が予定されています。Heraldは最初の実運用事例であり、66文書のマルチフォーマットコーパスを同期し、正常性を検証しています。

なぜ作ったのか

文書、エクスポート、データベースは、手作業や脆弱なスクリプトで管理されるとすぐに同期が崩れます。Docs Chainは同期を機械的かつコンテンツハッシュに基づく正確なものにし、チェーン内のどこかで変更があれば、下流（および上流）の全てを安全かつ正確に更新します。

コンテンツ

なぜゲームチェンジャーなのか

コンパイラやビルドシステムの開発者が当然のように享受してきた、苦勞して得た正確性の保証——コンテンツハッシュによる依存グラフ、最小限の再計算、アトミックなコミット——を、これまでcronジョブと善意に頼ってきたドキュメントやデータベースの領域に適用する。真の双方向同期により、ソースとそのエクスポートの関係性が双方向で強制されるため、「ドキュメントが古い」や「エクスポートがソースと一致しない」といった繰り返し発生するバグは、もはやエンジンが許容しない状態となる。

革新的な点

- コンテンツハッシュ（mtimeではなく）によるDAG上のインクリメンタル再計算と早期カットオフ。
- 双方向かつ権威宣言型の同期エッジ（ドキュメント ↔ エクスポート ↔ SQLite）。
- アトミックリネーム + SQLiteトランザクションによるクラッシュセーフな伝播。
- 純粋Goによるmd-to-sqlite/sqlite-to-mdラウンドトリップと行レベルのドリフト検出。

課題と解決策

- 不要な再ビルド：コンテンツハッシュ検出によりタイムスタンプに依存しない解決。
- 部分的／破損した更新：アトミックリネームとSQLiteトランザクションで解決。
- 複数メンバーの正しい順序付け：カーンのトポロジカルソート + 早期カットオフで解決。
- 正確な機能報告：各フェーズを§11.4.6に基づきIMPLEMENTEDまたはPLANNEDとして明示。

技術スタック（理由と方法）

- **Go** — エンジン全体（internal/hash、graph、adapter、orchestrator、config、state、runner、cmd/docs_chain）。
- **DAG +** カーンのトポロジカルソート — 依存関係の順序付けと早期カットオフ。
- **SQLite**（純粋Go modernc） — データベースメンバーとトランザクショナルコミット。
- **fsnotify** — ライブ伝播のためのwatchデーモン。
- **YAML**設定 — コンテキストごとのチェーン登録。
- **exec**: 変換 — プラガブルなMarkdown→HTML/PDF/DOCX生成。

ロードマップの透明性：フェーズ6～7（憲法配布、ATMOSphere接続）はPLANNED／オペレータ制限付きで提供 — まだリリースされていない。